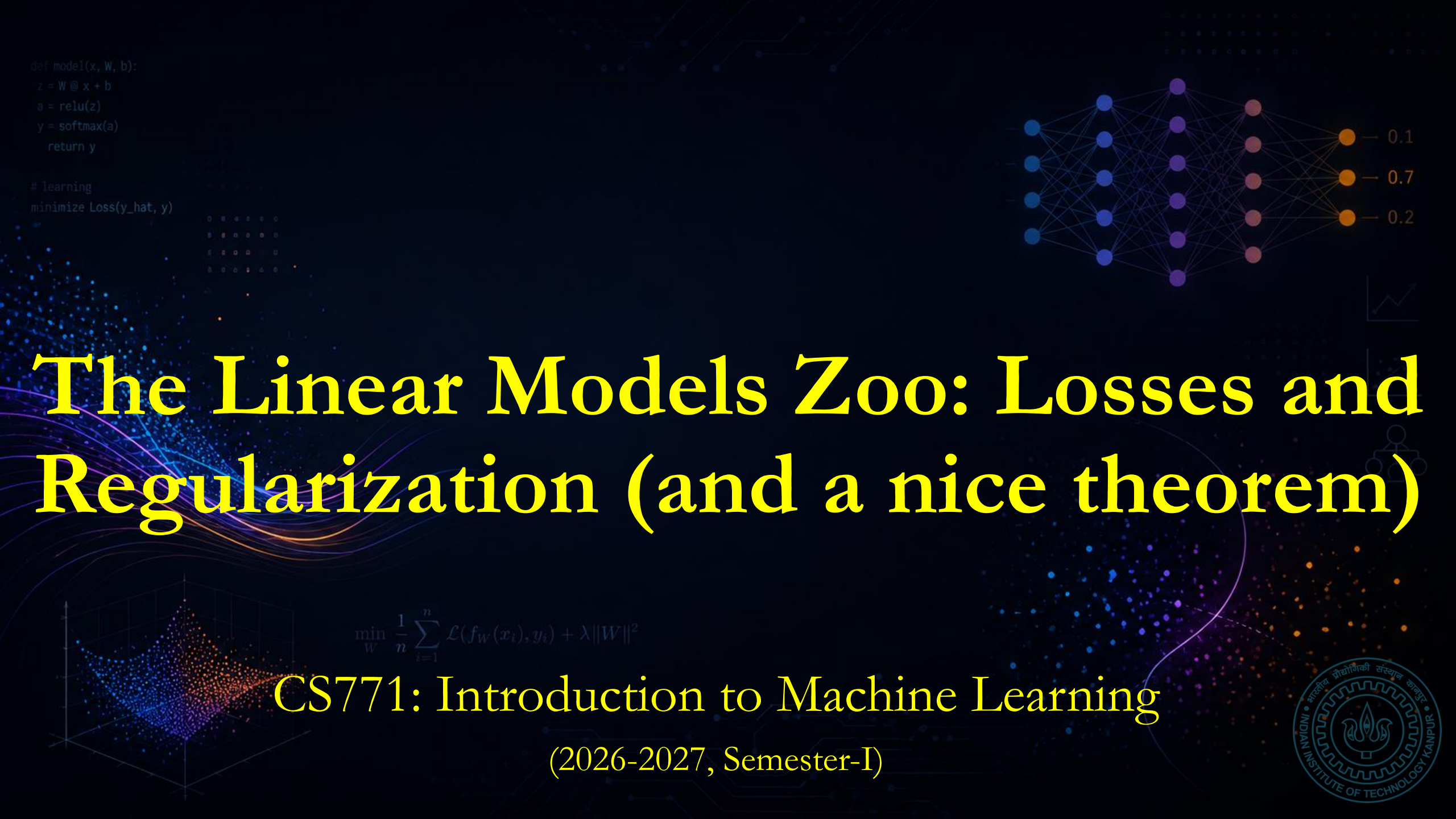


The Linear Models Zoo: Losses and Regularization (and a nice theorem)

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

One Linear Model, Useful for Many Problems

If using a feature transformation, replace $\boldsymbol{x}_n$ by $\boldsymbol{\phi}(\boldsymbol{x}_n)$

Given N training examples, loss function for a linear model may look like

$$\mathcal{L}(\boldsymbol{w}) = \sum_{n=1}^{N} \ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) \quad \text{or} \quad \mathcal{L}(\boldsymbol{W}) = \sum_{n=1}^{N} \ell(\boldsymbol{y}_n, \boldsymbol{W}^\top \boldsymbol{x}_n)$$

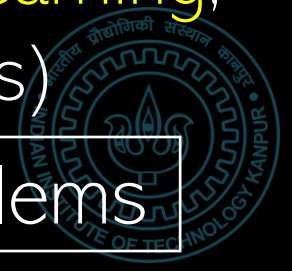
Note: Assumes $\boldsymbol{y}_n \in \mathbb{R}^C$ to be a vector and $\boldsymbol{W}$ to be a $D \times C$ matrix

Important: $\boldsymbol{w}^\top \boldsymbol{x}_n$ or $\boldsymbol{W}^\top \boldsymbol{x}_n$ is not model's *prediction* but model's "score" s_n for the input $\boldsymbol{x}_n$. This score is then transformed into the prediction, e.g.,

$$\hat{y}_n = s_n \qquad \hat{y}_n = \text{sign}(s_n) \qquad \hat{p}_n = \text{sigmoid}(s_n) \qquad \hat{\boldsymbol{p}}_n = \text{softmax}(s_n)$$

Linear models can be used for a wide variety of problems, e.g., regression, binary/multiclass/multi-label classification (and even unsupervised learning, such as representation learning, etc when training data has no labels)

Only the form of the loss function $\ell(.,.)$ changes across problems



The Linear Model Zoo: Different Loss Functions

Some popular examples of loss functions used within linear models

Squared loss (regression)

$$\ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) = (y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2$$

0-1 loss (binary classification)

$$\ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) = \mathbb{I}[y_n \neq \text{sign}(\boldsymbol{w}^\top \boldsymbol{x}_n)]$$

Absolute loss (regression)

$$\ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) = |y_n - \boldsymbol{w}^\top \boldsymbol{x}_n|$$

Logistic/log loss (binary classification) or binary cross-entropy loss

$$\ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) = \log(1 + \exp(-y_n \boldsymbol{w}^\top \boldsymbol{x}_n))$$

Softmax or cross-entropy loss (multi-class classification)

$$\ell(y_n, \boldsymbol{W}^\top \boldsymbol{x}_n) = -\boldsymbol{w}_{y_n}^\top \boldsymbol{x}_n + \log\left(\sum_{c=1}^{C} \exp(\boldsymbol{w}_c^\top \boldsymbol{x}_n)\right)$$

Note: In this case $y_n \in \{1,2,\ldots,C\}$ is a discrete (a.k.a. categorical) class label and $\boldsymbol{W} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_C]$ is a $D \times C$ matrix

Perceptron loss (binary classification)

$$\ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) = \max(0, -y_n \boldsymbol{w}^\top \boldsymbol{x}_n)$$

Hinge loss (binary classification) – used in support vector machines (SVM)

$$\ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) = \max(0, 1 - y_n \boldsymbol{w}^\top \boldsymbol{x}_n)$$

(.. and many variants of above losses (e.g., multi-class extensions), and losses for other problems that we will see later)

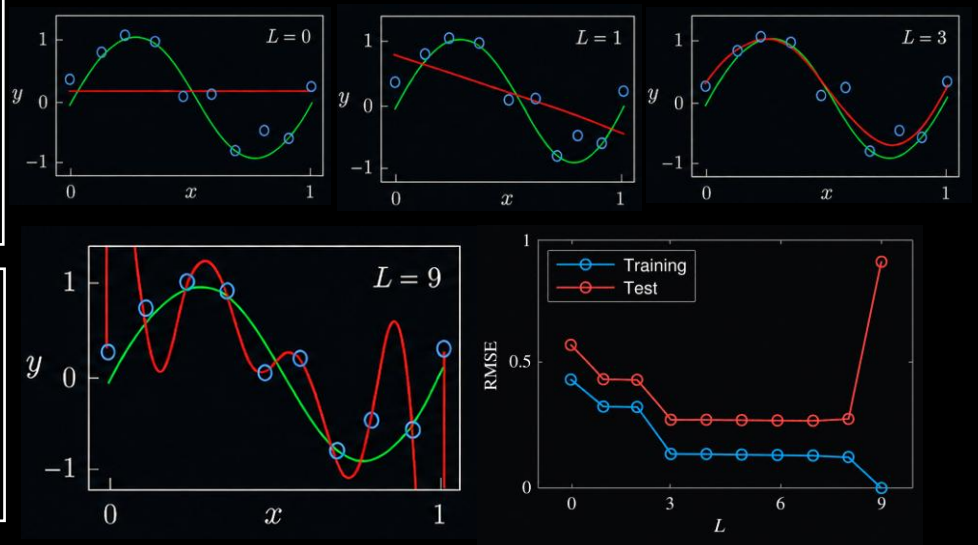
Is Training Loss Minimization Enough?

Consider a regression model with loss $\mathcal{L}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - (\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n) + b))^2$

Assuming each $\boldsymbol{x}_n$ to be scalar and a feature transformation $\boldsymbol{\phi}(x) = [x, x^2, \ldots, x^L]$

Minimizing $\mathcal{L}(\boldsymbol{w})$ will cause the weights to be adjusted with no control on their magnitudes, resulting in the learned function to become very complex, overfit, and perform poorly on test data

$L = 9$ (order 9 polynomial) achieves ZERO training loss .. but fails to learn that the true function is much smoother



	$L=0$	$L=1$	$L=6$	$L=9$
b (bias term)	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Magnitude of the weights (both individual and collectively) is a measure of the complexity of the learned model

Instead of minimizing just training loss, we should "regularize" the weights individually and/or collectively (ideally *both*) to control their magnitudes

INSTITUTE OF TECHNOLOGY KI

Norms to Measure Magnitude (and Regularize)

We can minimize norm of $\boldsymbol{w} \in \mathbb{R}^D$ to control its magnitude. Many choices:

ℓ_0 norm

$$\|w\|_0 = \mathbf{nnz}(\boldsymbol{w})$$

Counts the number of nonzero entries in $\boldsymbol{w}$

A very ideal measure of complexity of a weight vector.

But NP hard to minimize because it's akin to a subset selection problem ☹

Naturally tells which features are relevant

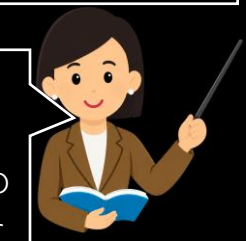
$w_i = 0$ means feature x_i (or $\phi_i(x)$) is not relevant



Wow, many choices! Which ones do we actually use?

Many theoretical results assume that the model has ℓ_2 squared norm as regularization

ℓ_2 squared norm is the de-facto choice but ℓ_1 is also used if we want $\boldsymbol{w}$ to be a sparse weight vector



ℓ_2 squared norm is the most common

$$\|\boldsymbol{w}\|_2^2 = \boldsymbol{w}^\top \boldsymbol{w} = \sum_{i=1}^{D} w_i^2$$

Minimizing the ℓ_2 norm promotes the individual entries of $\boldsymbol{w} \in \mathbb{R}^D$ to be small

Having small individual entries w_i in $\boldsymbol{w}$ is a good thing.

Recall the score is $s = \sum_{i=1}^{D} w_i x_i$
Small w_i's indicates that no single feature x_i (or $\phi_i(x)$) is allowed to dominate the prediction

ℓ_1 norm is also popular

$$\|\boldsymbol{w}\|_1 = \sum_{i=1}^{D} |w_i|$$

Minimizing ℓ_1 norm encourages $\boldsymbol{w}$ to be sparse vector (i.e., a vector with very few non-zero entries)

Similar in spirit to the ℓ_0 norm but much easier to optimize than ℓ_0 (and can be seen as a nice proxy of the ℓ_0 norm)

ℓ_∞ norm

$$\|w\|_\infty = \max(\{w_i\}_{i=1}^{D})$$

ℓ_P norm ($P \geq 1$)

$$\|w\|_p = \sum_{i=1}^{D} (|w_i|^P)^{1/P}$$

Models with Norm Based Regularizations

With a norm based regularization, we construct a regularized loss function

Assuming we have a linear model, the regularized loss function will be

Average training error achieved by $\boldsymbol{w}$

Regularization hyperparameter (a non-negative number, usually set via cross-validation)

Regularizer - a suitable norm function (e.g., ℓ_2, ℓ_1, etc) – penalizes large weights during optimization

$$\mathcal{L}_{reg}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} \ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) + \lambda\, R(\boldsymbol{w})$$

Minimizing $\mathcal{L}_{reg}(\boldsymbol{w})$ helps learn $\boldsymbol{w}$ that

- Achieves low training error
- Has small norm as measured by $R(\boldsymbol{w})$

There is an inherent trade-off between these two goals (can't have too small training error while also having very small norm)

λ is set to find the sweet-spot of this trade-off



Some Norm-Regularized Models for Regression

Assuming squared loss function and ℓ_2 squared regularizer

$$L_{reg}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \lambda \boldsymbol{w}^\top \boldsymbol{w}$$

The very popular ridge regression model

Assuming squared loss function and ℓ_1 regularizer

$$L_{reg}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \lambda \|w\|_1$$

The very popular Lasso regression model

Can be used if we want a sparse weight vector $\boldsymbol{w}$

Helps in automatically selecting the relevant features (recall that $w_i = 0$ means feature $\boldsymbol{\phi}_i(\boldsymbol{x})$ is irrelevant



A Closer Look at Norm Based Regularization

Note how the regularization hyprparameter λ acts on each entry of $\boldsymbol{w}$

$$\lambda\, R(\boldsymbol{w}) = \lambda\, \boldsymbol{w}^\top \boldsymbol{w} = \lambda \sum_{i=1}^{D} w_i^2$$

$$\lambda\, R(\boldsymbol{w}) = \lambda \|w\|_1 = \lambda \sum_{i=1}^{D} |w_i|$$

All w_i's penalized equally (extent controlled by the same regularization hyperparameter λ

Instead of a regularizer $\lambda\, R(\boldsymbol{w})$, we can have entry-specific regularization on $\boldsymbol{w}$ defined as $R(\boldsymbol{w}, \boldsymbol{\lambda})$ where $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \ldots, \lambda_D] \in \mathbb{R}_+^D$, e.g.,

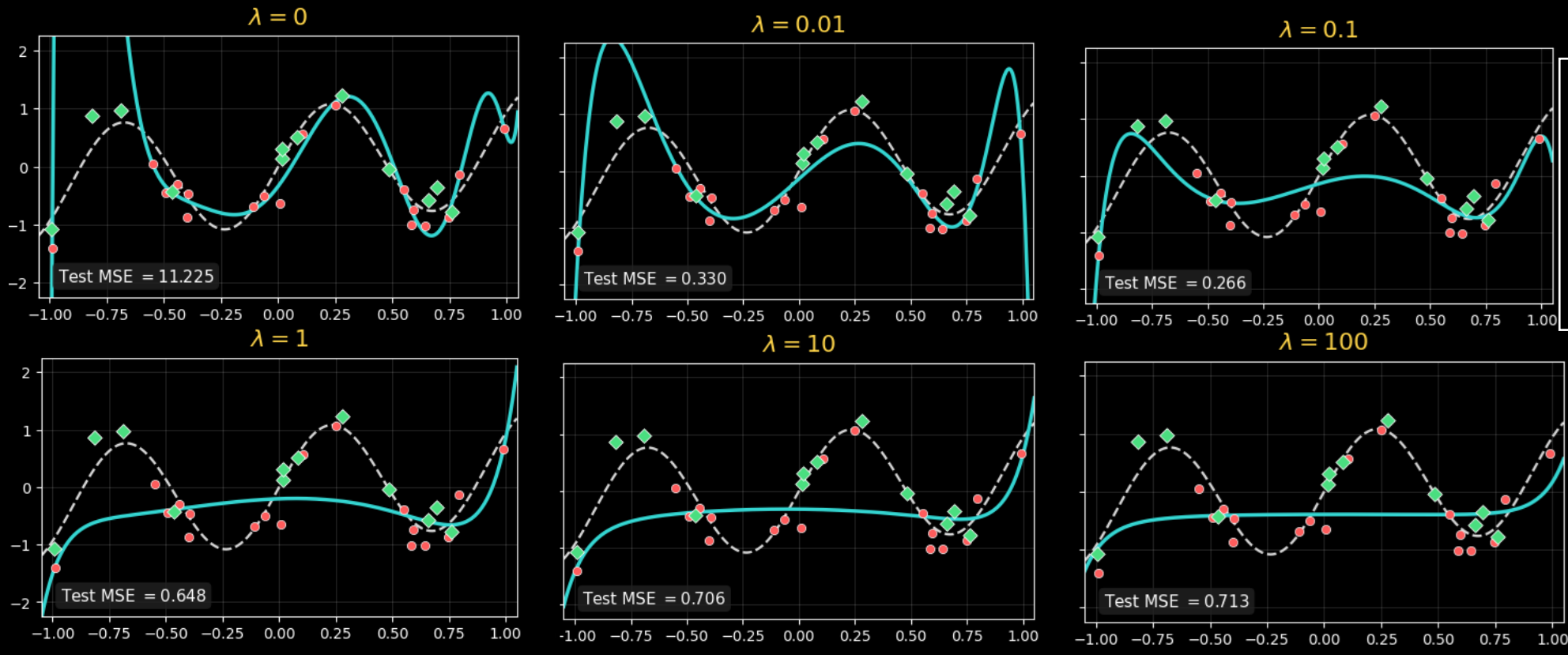
$$R(\boldsymbol{w}, \boldsymbol{\lambda}) = \sum_{i=1}^{D} \lambda_i\, w_i^2$$

$$\mathcal{L}_{reg}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, \boldsymbol{w}^\top \boldsymbol{x}_n) + R(\boldsymbol{w}, \boldsymbol{\lambda})$$



Greater Regularization = Smoother Models

$$\phi(x) = \begin{bmatrix} x \\ x^2 \\ \vdots \\ x^{15} \end{bmatrix} \qquad L_{reg}(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - (\boldsymbol{w}^\top \phi(x_n) + b))^2 + \lambda \boldsymbol{w}^\top \boldsymbol{w}$$





Very small λ: Almost no regularization. Very complex regression function – fits training data very well but doesn't do well on test data

Increasing λ: Regression function becomes more and more smooth.

Ideal λ (small training error and small test error) lies somewhere in between

Other Regularization Schemes

Norm based regularization is not the only way to regularize

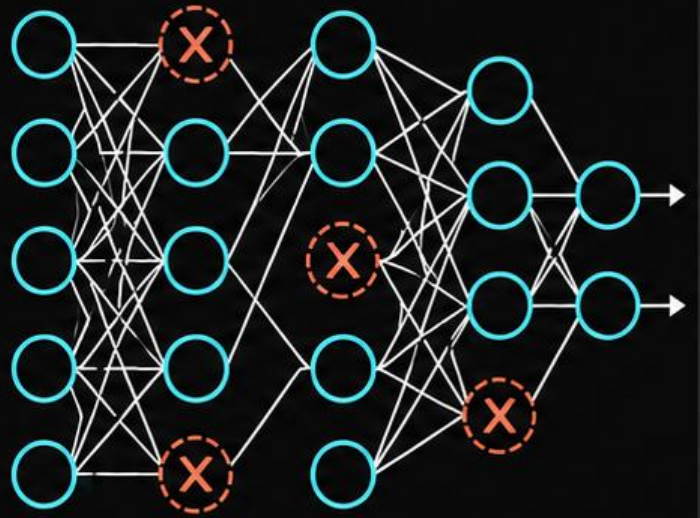
Other popular methods (uses especially in training of deep neural nets):

Early stopping



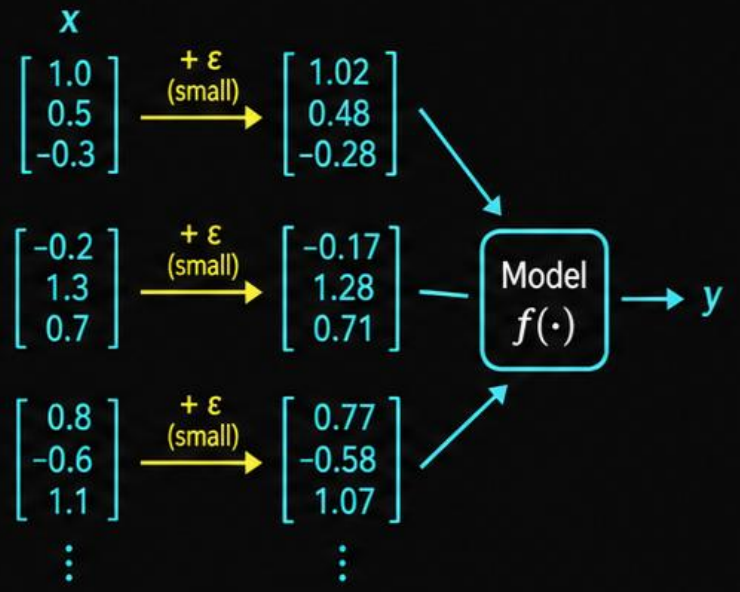
Stop before overfitting begins

Dropout



Train many thinned networks

Input noise



Learn robustness to small perturbations

The ML Recipe: Model Design + Optimization

Choose your model f, e.g., a linear model

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$$

For the chosen model, choose your loss function $\ell(.,.)$, e.g., squared loss

$$\ell(y, \boldsymbol{w}^\top \boldsymbol{x}) = (y - \boldsymbol{w}^\top \boldsymbol{x})^2$$

Choose a regularization scheme, e.g., norm based, early stopping, etc.

Choose the optimizer that is the most suitable for the above combination

The Model | Loss Function | Regularization Scheme | Optimizer

Making choices for these 3 aspects is usually considered part of the model design



A Theorem Expressing Optimal Weight Vector

Suppose $f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x})$ and consider the problem

$$\boldsymbol{w}_{\text{opt}} = \arg\min_w \left[\frac{1}{N} \sum_{n=1}^{N} (y_n - \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n))^2 + \lambda \boldsymbol{w}^\top \boldsymbol{w} \right]$$

Let's write $\boldsymbol{w} = \boldsymbol{w}_{\|} + \boldsymbol{w}_{\perp}$ where $\boldsymbol{w}_{\|} = \text{span}\{\boldsymbol{\phi}(\boldsymbol{x}_1), \boldsymbol{\phi}(\boldsymbol{x}_2), \ldots, \boldsymbol{\phi}(\boldsymbol{x}_N)\}$

$\boldsymbol{w}_{\|} = \sum_{n=1}^{N} \alpha_n \boldsymbol{\phi}(\boldsymbol{x}_n)$

.. and $\boldsymbol{w}_{\perp}$ is orthogonal to all the training inputs, i.e., $\boldsymbol{w}_{\perp}^\top \boldsymbol{\phi}(\boldsymbol{x}_n) = 0 \quad \forall n$

Thus $\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n) = \boldsymbol{w}_{\|}^\top \boldsymbol{\phi}(\boldsymbol{x}_n)$

So $\boldsymbol{w}_{\perp}$ doesn't affect the training loss, but only increases norm $\boldsymbol{w}^\top \boldsymbol{w}$

Thus the optimal $\boldsymbol{w}_{\text{opt}}$ must have $\boldsymbol{w}_{\perp} = \boldsymbol{0}$ and therefore $\boldsymbol{w}_{\text{opt}} = \boldsymbol{w}_{\|}$

$$\boldsymbol{w}_{\text{opt}} = \sum_{n=1}^{N} \alpha_n \boldsymbol{\phi}(\boldsymbol{x}_n)$$

A very fundamental result in machine learning

Popularly known as the Representer Theorem

A Unifying Form for Prediction Equations

Thanks to the Representer Theorem, we now know that

$$\boldsymbol{w}_{\text{opt}} = \sum_{n=1}^{N} \alpha_n \, \boldsymbol{\phi}(\boldsymbol{x}_n)$$

Similar results exist for other losses too (not just squared loss) and the optimal $f(.)$ in those cases too can be written as a span of training inputs

Thus the prediction ("score") for any test input, say $\boldsymbol{\phi}(\boldsymbol{x}_*)$, can be written as

$$s_* = f(\boldsymbol{\phi}(\boldsymbol{x}_*)) = \boldsymbol{w}_{\text{opt}}^{\top} \boldsymbol{\phi}(\boldsymbol{x}_*)$$

$$= \sum_{n=1}^{N} \alpha_n \, \boldsymbol{\phi}(\boldsymbol{x}_n)^{\top} \boldsymbol{\phi}(\boldsymbol{x}_*)$$

This is nothing but a dot-product based similarity using the $\boldsymbol{\phi}(.)$ representation of the inputs

$$= \sum_{n=1}^{N} \alpha_n \, k(\boldsymbol{x}_n, \boldsymbol{x}_*)$$

Thus the prediction/score of a test input is a weighted sum of its similarities with each of the N training inputs where the similarity is defined via a suitable function $k(\boldsymbol{x}_n, \boldsymbol{x}_*) = \boldsymbol{\phi}(\boldsymbol{x}_n)^{\top} \boldsymbol{\phi}(\boldsymbol{x}_*)$ (such as an RBF kernel function or others)



The Representer Theorem result $\boldsymbol{w}_{\text{opt}} = \sum_{n=1}^{N} \alpha_n \, \boldsymbol{\phi}(\boldsymbol{x}_n)$ and the result about the form of the predictions $s_* = \sum_{n=1}^{N} \alpha_n \, k(\boldsymbol{x}_n, \boldsymbol{x}_*)$ are fundamental results central to much of ML, and provide a unifying view to many ML models where the optimization objective can be written as a sum of a loss function and regularizer

A Primal-Dual Perspective to Learning

The Representer Theorem told us that the solution for $\boldsymbol{w} \in \mathbb{R}^D$ is

$$\boldsymbol{w}_{\mathbf{opt}} = \sum_{n=1}^{N} \alpha_n \, \phi(\boldsymbol{x}_n)$$

So we can optimize explicitly for the D weights $\boldsymbol{w} = [w_1, w_2, \ldots, w_D]^\top$ (One per input feature)

.. or optimize N coefficients $\boldsymbol{\alpha} = \{\alpha_1, \alpha_2, \ldots, \alpha_N\}$ (One per training input)

Both approaches would give the same solution for $\boldsymbol{w}_{\mathbf{opt}}$ but have trade-offs

Some considerations for which method to choose:

- If $N \gg D$, optimize the weights w_i's (called the "primal form"
- If $D \gg N$, optimize the coefficients α_n's (called the "dual form")
- Prediction is usually faster in the primal form ($O(D)$ scalar multiplications)

